

Programmeerproject 1
Verslag “Fire Ant”

Academiejaar 2021-2022



Naam: Gürkan Hamarat
Mail: gurkan.hamarat@vub.b
Rolnummer: 0588505

Inhoudsopgave

1	Introductie	3
2	ADTs	4
2.1	positie-adt.....	4
2.2	Constanten ADT.....	5
2.3	Hulp-procedures ADT.....	5
2.4	Spel ADT.....	5
2.5	voedsel-adt.....	5 en 6
2.6	spel-adt.....	6
2.7	Doolhof-adt.....	7
2.8	vuurmier-adt.....	7 en 8
2.9	schorpioen-adt.....	8
2.10	level-adt.....	9
2.11	draw-adt.....	10
3	Hoe spel opstarten?	10
4	Afhankelijkheidsdiagram	11
4.1	afhankelijkheidsdiagram (voorstudie).....	11
4.2	afhankelijkheidsdiagram (verslag).....	12 en 13
5	Planning	14
5.1	planning (voorstudie).....	14
5.2	planning (verslag).....	14

1 Introductie

In dit document wordt een manier getoond om het spel “Fire Ant” te implementeren in de programmeertaal Scheme. Deze taak behoort tot het vak “Programmeerproject 1”. Om het spel te laten functioneren moeten verschillende stappen ondernomen worden. Er moeten elementen geïmplementeerd worden die ervoor zorgen dat elk uniek onderdeel van het spel juist funcioneert. Het implementatie van de spel wordt gedaan in twee fases. In de eerste fase wordt er gevraagd om elementaire componenten en handelingen te implementeren. Hierbij wordt de basisbenodigdheden om het spel te kunnen spelen in acht genomen:

- Een spelwereld dat bestaat uit een doolhof en een vuurmier. De vuurmier wordt bestuurd door de speler.
- Minstens één schorpioen die een vast pad volgt met een begin- en eindpunt. In de eerste fase zijn er nog geen levens. Er zijn eitjes verspreid doorheen het level. In de eerste fase verdwijnen de eitjes bij aanraking, maar de score wordt nog niet verhoogd.

Dit verslag bevat de beschrijving van de code nodig voor de functionaliteit van fase 1. Er werd hiervoor gebruik gemaakt van 11 ADT's : positie-adt Constanten ADT, Hulp-procedures ADT, Spel, spel-adt, Doolhof-adt, vuurmier-adt, voedsel-adt, level-adt, schorpioen-adt, draw-adt. Voor Uitleg over de verschillende ADTs zie verslag vanaf pagina 4.

In de tweede fase zijn er dan extra elementen die toegevoegd worden:

- Meerdere schorpioenen in elk level van beide types. Er zijn hierbij schorpioenen die levens afnemen van de vuurmier wanneer die de schorpioen aanraakt en er is ook een nieuwe functionaliteit dat schorpioenen op willekeurige momenten sneller doet bewegen.
- Minstens twee soorten puzzels uit de lijst en ook andere die je zelf kan bedenken.
- Het ondersteunen van verschillende soorten voedsel objecten. Deze voedsel objecten

moeten een effect hebben op de score.

- Het bijhouden van de score, de hoogste score en het aantal levens.
- Minstens twee soorten power-ups uit de lijst en ook andere die je zelf kan bedenken.
- Minstens drie levels die elk een ander doolhof hebben met andere puzzels.

Dit document is verdeeld in vier verschillende hoofdstukken. De eerste hoofdstuk is de introductie, de tweede hoofdstuk bevat de ADTs die gebruik worden tijdens het project, de derde hoofdstuk geeft in een schema weer welke ADTs met elkaar verbonden zijn en ten slotte geeft de laatste hoofdstuk de planning weer tijdens het uitvoeren van het project.

De code voor fase 1 werd zodanig geïmplementeerd zodat fase 2 makkelijk te implementeren is en waarbij geen code veranderd moet worden aan die van fase 1.

2 ADTs

Zoals vermeld in de introductie gaat de tweede hoofdstuk over de geïmplementeerde ADTs.

2.1 positie-adt

Het positie-adt is nodig voor het behouden van een positie met twee coördinaten. Er wordt een x- en y-waarde gegeven aan een element binnen het spel. Het positie-adt voorziet enkel operaties die toegepast kunnen worden op x- en y-coördinaten. De positie-adt is handig en kan gebruikt worden binnen andere ADTs. Dit ADT is nodig om posities aan te maken en om ze te manipuleren.

Naam	Signatuur
maak-positie	(number number $\rightarrow$ positie)
x	($\emptyset \rightarrow$ number)
y	($\emptyset \rightarrow$ number)
x!	(number $\rightarrow \emptyset$)
y!	(number $\rightarrow \emptyset$)
gelijk?	(positie $\rightarrow$ boolean)
beweeg!	(number number $\rightarrow \emptyset$)

Operaties van het positie-adt :

- maak-positie is de operatie die het positie-adt aanmaakt. Om een object van het positie-adt aan te maken moeten er twee getallen meegegeven worden. Deze twee getallen stellen na de maak-positie operatie een positie voor. Zo kan je aan elk voorwerp (bijvoorbeeld schorpioenen) binnen je spel een coördinaat geven.
- x en y zijn de operaties om de x- en y-waarde van een voorwerp aan te vragen.
- x! en y! zijn operaties om de x- en y-waarde van een voorwerp aan te passen. Hiermee moeten nieuwe waarden gegeven worden en zo zullen de coördinaten veranderd worden.
- gelijk? Is een operatie die de coördinaten vergelijkt van twee voorwerpen en het geeft #t weer als de twee voorwerpen dezelfde coördinaten hebben, anders geeft het #f weer. De operatie beweeg zorgt ervoor dat een voorwerp zal bewegen.

Alle ADT's behalve met Hulp-procedures ADT interageren met positie-adt.

2.2 Constanten

Het Constanten ADT wordt gebruikt om constanten die belangrijk zijn een naam te geven.

Met belangrijk bedoel ik constanten die meerdere keren gebruikt zullen worden. Er zijn momenteel 9 constanten. Voor deze ADT is er geen tabel met proceduretypes omdat het eigenlijk een ADT is voor abstractie, er worden geen nieuwe procedures geïmplementeerd, het is een ADT om gewoon waarden een naam te geven.

Alle ADT's behalve Hulp-procedures ADT, positie-adt en Doolhof-adt interageren niet met Constanten ADT.

2.3 Hulp-procedures

Dit ADT is nodig om hulp-procedures te kunnen gebruiken die van nut zullen zijn.

Naam	Signatuur
debugging?	$(\emptyset \rightarrow \text{boolean})$
debug	$(\text{any} \rightarrow \emptyset)$

Operaties van het Hulp-procedures ADT:

- debugging geeft een boolean weer. Debugging is gelijk aan #t.
- debug is een procedure die de gegeven parameter zal display als het een true weergeeft.

Hulp-procedure ADT interageert momenteel met geen andere ADT.

2.4 Spel

Dit ADT wordt gebruikt om alle andere ADT's te verbinden. Bijvoorbeeld om procedures van positie-adt te kunnen gebruiken in Doolhof-adt. Er is een andere ADT genaamd spel-adt. In het ADT Spel wordt maak-adt-spel opgeroepen die afkomstig is van spel-adt. Deze procedure is hetgene dat het spel start. Wanneer er op run wordt gedrukt op het ADT Spel, wordt het spel gestart.

Spel ADT zorgt ervoor dat alle ADT's tussen elkaar kunnen gebruikt worden dus alle ADT's interageren met Spel ADT.

2.5 voedsel-adt

Het voedsel-adt is nodig om voedsels binnen het spel, die kunnen opgeraapt worden door de vuurmier, te implementeren. Hierbij hebben ze een relatie met de vuurmier, omdat deze voedsels verdwijnen van het spel wanneer ze aangeraakt worden door de vuurmier. In de eerste fase zijn er alleen nog maar de eitjes. Ik geef nu dit ADT de naam voedsel-adt zodat ik in fase twee andere elementen kan toevoegen binnen dit ADT (zoals bijvoorbeeld de puzzels) en de naam van het ADT dan niet hoeft aan te passen.

Naam	Signatuur
maak-voedsel	$(\text{positie} \rightarrow \text{voedsel})$
Positie	$(\emptyset \rightarrow \text{positie})$
positie!	$(\text{positie} \rightarrow \emptyset)$

Operaties van het voedsel-adt:

- maak-voedsel is de operatie die het voedsel-adt aanmaakt. Voor een object van het voedsel-adt aan te maken moet er een positie van het voedsel (ei) meegegeven worden.
- positie! en positie zijn operaties die kunnen gebruikt worden om de positie van de stuk voedsel aan te passen en om de huidige positie van de karakter aan te vragen.

De ADT' die met voedsel-objecten interageert is Level ADT, Draw ADT en spel-adt.

2.6 spel-adt

Het spel-ADT is het ADT dat ervoor zorgt dat wanneer er een toets wordt ingedrukt het spel weet wat er gedaan moet worden, bijvoorbeeld linkerpijl laat de vuurmier links bewegen. Het is een ADT dat ervoor zorgt dat er steeds een lus wordt gemaakt, alles wordt steeds getekend, wanneer vuurmie beweegt wordt dit getoond op de window.

Naam	Signatuur
Maak-adt-spel	$(\emptyset \rightarrow \text{spel})$
Toets-procedure	$(\text{symbol any} \rightarrow \emptyset)$
Spel-lus-procedure	$(\text{number} \rightarrow \emptyset)$
Start	$(\emptyset \rightarrow \emptyset)$

Operaties va het spel-adt:

- maak-adt-spel is de operatie die het Spel-adt aanmaakt. Voor een object van het spel-adt aan te maken moet er niets meegegeven worden.
- Toets-procedure is de operatie die aan de hand van of er een toets word ingedrukt wordt ervoor zal zorgen dat er iets gebeurd op de window, bijvoorbeeld met rechterpijl zal de vuurmier naar rechts bewegen indien mogelijk.
- Spel-lus-procedure is de operatie die ervoor zorgt dat steeds alles getekend wordt en uitgevoerd wordt
- start operatie is de operatie die het spel zal opstarten.

Het Spel ADT interageert met spel-adt (namelijk met 'start').

2.7 Doolhof-adt

Dit is het ADT dat het onderscheid zal maken tussen de verschillende doolhoven die in de verschillende levels geïmplementeerd zullen worden. In dit ADT maak ik een doolhof door een vector van vectoren aan te maken.

Naam	Signatuur
maak-doolhof-adt	(symbol $\rightarrow$ doolhof)
waarde?	(doolhof number number $\rightarrow$ number)
Doolhof	($\emptyset \rightarrow$ Doolhof)
beweegt!	(symbol (number number $\rightarrow$ positie) $\rightarrow$ positie)

Operaties van het Doolhof-adt:

- maak-doolhof-adt is de operatie die het Doolhof-adt aanmaakt.
- waarde? is de operatie die kijkt wat de waarde is van de element op een bepaalde plaats in de vector van vectoren (doolhof)
- Doolhof is de operatie gewoon de doolhof weergeeft die is gemaakt om aan te duiden waar vuurmier en schorpioen kunnen bewegen en waar niet.
- De beweegt! operatie is nodig, met deze operatie ga je na wat de element is op een bepaald positie, als het element 0 is op de volgende positie dan blijf je op je huidige positie en als het element een 1 is mag je naar de volgende positie gaan.

De ADT's die met doolhof objecten interageren, zijn vuurmier-adt en schorpioen-adt. Ze maken gebruik van beweegt! om ervoor te zorgen dat de schorpioen en vuurmier alleen maar in de gangen van de doolhof bewegen en niet op de muren.

2.8 vuurmier-adt

Dit is het ADT dat ervoor zorgt dat de vuurmier beweegt in de gang van de doolhof. In dit ADT wordt de positie van de vuurmier op elk moment bijgehouden. Je kan ook de positie veranderen aan de hand van vuurmier-adt.

Naam	Signatuur
Maak-vuurmier-adt	(positie $\rightarrow$ vuurmier)
positie	($\emptyset \rightarrow$ positie)
Positie!	(positie $\rightarrow \emptyset$)
Vuurmier-beweegt!	(symbol $\rightarrow$ positie)

Operaties van vuurmier-adt het :

- maak-vuurmier -adt is de operatie die het vuurmier-adt aanmaakt. Voor een object van het vuurmier-adt aan te maken moet er een positie meegegeven worden.
- positie! en positie zijn operaties die kunnen gebruikt worden om de positie van de vuurmier aan te passen en om de huidige positie van de vuurmier aan te vragen.

- Vuurmier-beweegt! zorgt ervoor dat de vuurmier kan bewegen naar links, rechts,omhoog en omlaag, zonder de muren te overlappen en alleen in de gangen te blijven.

Alle ADT's behalve Hulp-procedures ADT, positie-adt ,Doolhof-adt en Constanten ADT interageren met vuurmier objecten.

2.9 schorpioen-adt

Dit is het ADT dat ervoor zorgt dat de schorpioen beweegt in de gang van de doolhof. In dit ADT wordt de positie van de schorpioen op elk moment bijgehouden. Je kan ook de positie veranderen aan de hand van schorpioen-adt. Verschillend met de vuurmier is dat er bij een schorpioen de richting ook belangrijk is doordat het niet door de speler wordt bediend. Momenteel gaat de vuurmier van rechts naar links, daarom is de richting van belang.

Naam	Signatuur
maak-schorpioen-adt	(positie→ schorpioen)
positie	($\emptyset$ → positie)
positie!	(positie → $\emptyset$)
schorpioen-beweegt!	(symbol→positie)
set-richting!	(symbol→symbol)
richting-geven	($\emptyset$ → symbol

Operaties van schorpioen-adt:

- maak-schorpioen-adt is de operatie die het schorpioen-adt aanmaakt.Voor een object van het schorpioen-adt aan te maken moet er een positie meegegeven worden.
- positie! en positie zijn operaties die kunnen gebruikt worden om de positie van de schorpioen aan te passen en om de huidige positie van de schorpioen aan te vragen.
- schorpioen-beweegt! is de operatie die ervoor zorgt dat dat de schorpioen beweegt en dat het de muren niet overlapt.
- Richting-geven geeft de richting weer die de schorpioen op dat ogenblik volgt.

Alle ADT's behalve Hulp-procedures ADT, positie-adt ,Doolhof-adt , Constanten ADT en vuurmier-adt interageren met schorpioen objecten.

2.10 level-adt

Dit ADT is zeer belangrijk, het zorgt ervoor dat de spellogica goed getekend wordt.

Naam	Signatuur
Maak-adt-level	(number number $\rightarrow$ level)
Toets!	(symbol $\rightarrow$ positie)
Update !	(number $\rightarrow \emptyset$)
Vuurmier	($\emptyset \rightarrow$ vuurmier)
Vuurmier-start-positie	($\emptyset \rightarrow$ positie)
Schorpion	($\emptyset \rightarrow$ schorpion)
Schorpion-start-positie	($\emptyset \rightarrow$ positie)
voedsel1	($\emptyset \rightarrow$ voedsel)
Voedsel2	($\emptyset \rightarrow$ voedsel)
Vergelijk	($\emptyset \rightarrow$ boolean)
Vuurmier-raakt-schorpion	($\emptyset \rightarrow$ positie)
Vuurmier-eet-appel	($\emptyset \rightarrow$ positie)

Operaties van level-adt:

- maak-adt-level is de operatie die het level-adt aanmaakt.
- Toets! is de operatie die de vuurmier laat draaien.
- Update! is de operatie om telkens de procedures opnieuw op te roepen zodat steeds alles getekend wordt op de window op een correcte manier.
- Vuurmier staat voor vuurmier-adt en het zorgt ervoor dat procedures uit vuurmier-adt in level-adt gebruikt kunnen worden, hierbij is de positie meegegeven aan maak-vuurmier-adt en stelt vuurmier-adt een aangemaakte vuurmier voor.
- vuurmier-start-positie geeft de start positie van de vuurmier weer
- schorpion staat voor schorpion-adt en het zorgt ervoor dat de procedures uit schorpion-adt in level-adt gebruikt kunnen worden, hierbij is de positie meegegeven aan maak-schorpion-adt en stelt schorpion-adt een aangemaakte schorpion voor.
- Schorpion-start-positie geeft de start positie van de schorpion weer.
- Voedsel1 staat voor voedsel-adt1 die de eerste ei voorstelt in het spel.
- Voedsel2 staat voor voedsel-adt2 die de tweede geïmplementeerde ei voorstelt.
- Vuurmier-raakt-schorpion is de operatie die ervoor zorgt dat wanneer de vuurmier de schorpion aanraakt of omgekeerd, dat dan de vuurmier naar een veilige plaats gaat (zijn start-positie).
- Vuurmier-eet-appel is de operatie die ervoor zorgt dat de appel verdwijnt bij aanraking door de vuurmier.

Draw-adt en spel-adt interageren met de level-objecten.

2.11 Draw ADT

Draw ADT is het ADT die de tekenlogica apart houdt van de spellogica.

Naam	Signatuur
Maak-draw-adt	(number number $\rightarrow$ draw)
Set-toets-functie!	(symbol any $\rightarrow \emptyset$)
Set-spel-lus-functie!	(number $\rightarrow \emptyset$)
Teken-spel	(spel $\rightarrow \emptyset$)

Operaties van draw-adt:

- maak-draw-adt is de operatie die de draw-adt zal aanmaken.
- Set-toets-functie! is de operatie die set-key-callback oproept binnen zijn body.
- set-spel-lus-functie is de operatie die set-update-callback oproept binnen zijn body.
- Teken-spel is de operatie die ervoor zorgt dat alles wordt getekend op de window zoals meegegeven in level-adt.

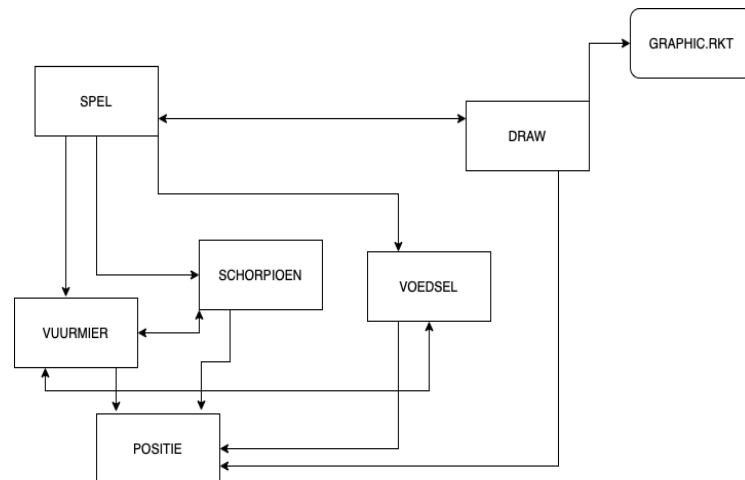
Spel-adt interageert met draw-adt

3 Hoe spel opstarten?

Nadat u de zip-bestand krijgt moet u Spel.rkt openen in Dr.Racket en dan op run drukken. Hiermee zal de spel gestart worden en krijgt u een window te zien. De groene cirkel is de vuurmier, de rode cirkel is de schorpioen en de appel stelt de ei voor.

4 Afhankelijkheidsdiagram

4.1 afhankelijkheidsdiagram (voorstudie)

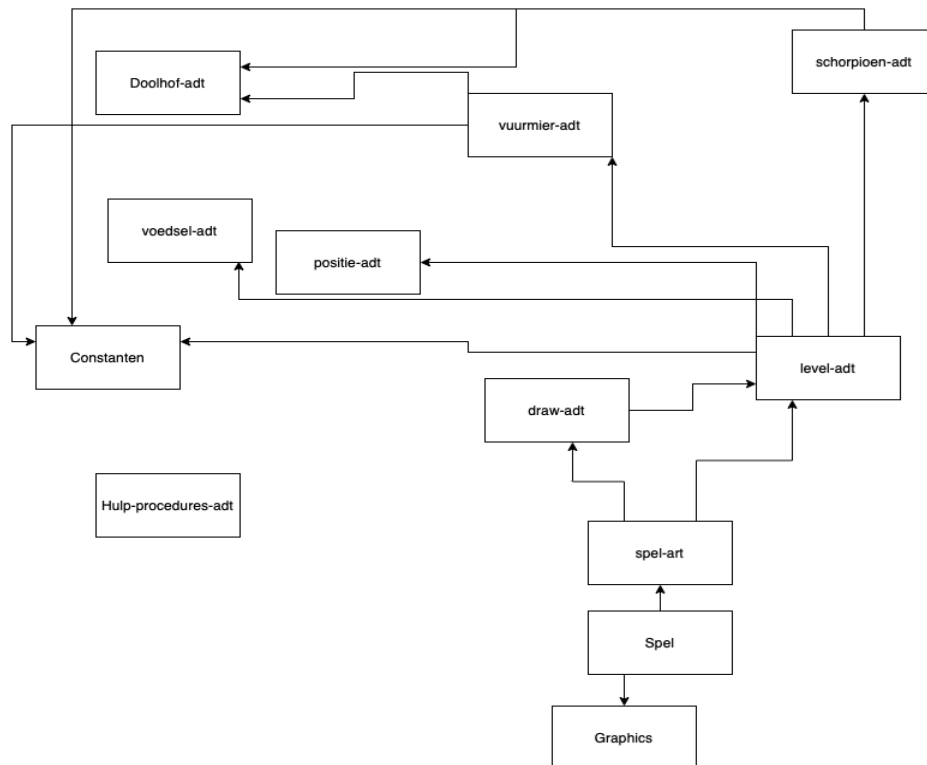


- het *Spel ADT* is afhankelijk van het *Vuurmier ADT*, *Schorpioen ADT* en het *Voedsel ADT* omdat dit de drie spelelementen zijn die aangestuurd worden vanuit het *Spel ADT*.
- Daarnaast is het *Spel ADT* ook afhankelijk van het *Draw ADT* om de spelsituatie te kunnen tekenen op het scherm.
- Het *Draw ADT* zelf is ook afhankelijk van het *Spel ADT* omdat het *Draw ADT* informatie aan het *Spel ADT* opvraagt om de huidige spelsituatie te kunnen tekenen.
- Zowel het *Vuurmier ADT* als het *Schorpioen ADT* als het *Voedsel ADT* zijn afhankelijk van het *Positie ADT* omdat ze een positie bijhouden.
- Het *Draw ADT* is het enige ADT dat afhankelijk is van de grafische bibliotheek (aangeduid als *Graphics.rkt* in het diagram). Het is het enige ADT dat gebruikt maakt van de functionaliteit van de grafische bibliotheek.
- Het *Draw ADT* is afhankelijk van het *Positie ADT*. De Positie van de verschillende karakters binnen het spel, zoals de vuurmier, moet steeds getekend en weergegeven worden op de window(venster).
- Het *Vuurmier ADT* is afhankelijk van het *Schorpioen ADT* en het *Voedsel ADT* omdat wanneer de vuurmier deze elementen aanraakt, er iets moet gebeuren. Bij aanraking tussen

de vuurmier en de schorpioen zal de vuurmier in de eerste fase alleen nog maar verplaatst worden naar een veilige plaats en bij aanraking tussen de schorpioen en de eitjes gebeurt er in deze fase niets, maar later in fase twee zullen de eitjes een effect hebben op

- Het *Schorpioen ADT* is ook afhankelijk van het *Vuurmier ADT* omdat bij aanraking de schorpioen ervoor zorgt dat de vuurmier van positie verandert.
- Het *Voedsel ADT* is ook afhankelijk van de *Vuurmier ADT* omdat bij aanraking tussen het voedsel (alleen eitjes in fase één) en de vuurmier, het voedsel verdwijnt van het spel.

4.2 afhankelijkheidsdiagram (verslag)



- Hulp-procedures-adt is in deze fase onafhankelijk van andere ADT's en er zijn ook geen ADT's afhankelijk van Hulp-procedures-adt.
- Het Spel ADT is afhankelijk van graphics omdat (`#%require "Graphics.rkt"`) in het Spel ADT is.
- Het Spel-ADT is afhankelijk van het spel-adt doordat er een object van spel-adt wordt gebruikt in Spel ADT, namelijk start.
- spel-adt is afhankelijk van level-adt doordat in spel-adt level-objecten aanwezig zijn die gebruikt worden in de body van procedures in spel-adt, bijvoorbeeld update! .
- spel-adt is afhankelijk van draw-adt doordat in spel-adt draw-objecten aanwezig zijn die gebruikt worden in de body van procedures in spel-adt, bijvoorbeeld teken-spel! .
- Level-adt is afhankelijk van positie-adt doordat er nieuwe posities worden gevormd in level-adt en hierbij wordt er gebruik gemaakt van maak-positie dat in positie-adt afkomstig is.
- Level-adt is afhankelijk van voedsel-adt doordat er voedsel-objecten zijn die gebruikt worden in level-adt, bijvoorbeeld maak-voedsel-adt.
- Level-adt is afhankelijk van vuurmier-adt doordat er vuurmier-objecten zijn die gebruikt worden in level-adt, bijvoorbeeld vuurmier-beweegt! .
- Level-adt is afhankelijk van schorpioen-adt doordat er schorpioen-objecten zijn die gebruikt worden in level-adt, bijvoorbeeld beweeg! .
- Level-adt is afhankelijk van Constanten ADT doordat er constanten zijn die gebruikt worden in level-adt die gedefinieerd waren in Constanten ADT , bijvoorbeeld schorpioen-snelheid.
- Draw-adt is alleen afhankelijk van level-adt doordat er objecten uit level-adt zijn die

gebruikt worden in draw-adt zoals bijvoorbeeld voedsel2.

- Schorpioen-adt is afhankelijk van doolhof-adt doordat de schorpioen niet op de muren van de doolhof mag rondlopen. Hiervoor is er een procedure beweegt! In doolhof-adt die gebruikt wordt in schorpioen-adt
- Er is een constante, start-positie-schorpioen, die is gedefinieerd in Constanten ADT en die wordt gebruikt in schorpioen-adt. Hierdoor is schorpioen-adt afhankelijk van Constanten ADT.
- Vuurmier-adt is afhankelijk van doolhof-adt doordat de vuurmier niet op de muren van de doolhof mag rondlopen. Hiervoor is er een procedure beweegt! In doolhof-adt die gebruikt wordt in vuurmier-adt
- Er is een constante, start-positie-vuurmier, die is gedefinieerd in Constanten ADT en die wordt gebruikt in vuurmier-adt. Hierdoor is vuurmier-adt afhankelijk van Constanten ADT.
- Doolhof-adt is afhankelijk van positie-adt doordat de procedure maak-positie wordt gebruikt binnen doolhof-adt.

Conclusie:

ik heb bijna alles anders gedaan dan wat ik dacht te doen terwijl ik mijn voorstudie schreef. Ik dacht niet dat voor fase 1 ik zo veel ADT's nodig had en dat ik zo veel lijnen code moest schrijven, ik heb fase 1 een beetje onderschat.

5 Planning

5.1 planning (voorstudie)

Week	Actie
Week 10	Afwerken en indienen voorstudie
Week 11	Implementeren van het Positie ADT
Week 12	Basis voorzien van het Draw ADT en het Spel ADT
Week 13	Implementatie van het Schorpioen ADT om één schorpioen te laten bewegen
Week 14	Implementatie van het Vuurmier ADT om de vuurmier te laten bewegen
Week 15-16	Afwerken Spel ADT en implementatie van het Voedsel ADT
Week 17-20	Examenperiode
Week 21-22	Verslag schrijven
Week 23	Indienen project en verslag

Tabel 7: Mijn planning voor het computerspel “Fire Ant” te implementeren.

5.2 planning (verslag)

Week	Actie
Week 10	Afwerken en indienen voorstudie
Week 11	Implementeren van het positie-adt
Week 12	Afronden positie-adt en implementatie Hulp-procedures-adt en Constanten ADT
Week 13	Implementatie spel-adt en Spel AFT
Week 14	Implementatie doolhof-adt
Week 15-16	examen-studeren
Week 17-20	examenperiode
Week 21-22	Vuurmier-adt, Draw-adt, level-adt
Week 23	Voedsel-adt, schorpioen-adt

De tweede tabel is wat ik uiteindelijk heb gedaan voor mijn project en de eerste is wat ik wou doen.

Conclusie: Ik ben goed begonnen, maar wanneer de examenperiode dichterbij kwam deed ik iets aan mijn project om zo te studeren, hierdoor heb ik op het einde meer werk gehad. Ik moet me beter organiseren om zo te kunnen studeren en te kunnen werken aan mijn project.